

# **Embedded System**

## **High Level Hardware Synthesis**

**Mostafiz Ahammed**

*Lecturer*

*Department of IoT and Robotics Engineering  
University of Frontier Technology, Bangladesh.*

# Topics

- Mapping Specification → HW by High Level Synthesis / Design (architecture synthesis for data-dominated systems)
  - High Level Synthesis Overview
  - **Basic Scheduling Algorithms: ASAP, ALAP**
  - Resource-Constrained Scheduling
  - Time-Constrained Scheduling
  - Advanced Scheduling: Operator Pipelining, Chaining & Multicycling
  - Allocation, Binding, Data Path & Controller Generation
  - Pipelined Data Path Design
  - Pipeline Controller Design

# Field Programmable Gate Arrays (FPGA)

- An FPGA (Field-Programmable Gate Array) is a flexible integrated circuit (IC) that users can program or reconfigure *after* manufacturing, unlike fixed-function chips.
- It contains an array of programmable logic blocks and interconnects, allowing designers to create custom digital circuits for various applications, from prototyping to high-performance computing (HPC), AI, telecommunications, and automotive systems, offering hardware-level performance with software-like adaptability.
- Components of FPGA
  - Flip-Flops (FF), small memory component able to store a bit
    - ✓ Typical used as a fast register to store data
  - Look-Up Tables (LUT), small memories used to store truth tables and perform logic functions
    - ✓ Typically used to perform operation such as “and”, “or”, sums or subtractions
  - Digital Signal Processor (DSP), small processor able to quickly perform mathematical operation on streaming digital signals
    - ✓ Typically used for multiplication and additions
  - Block RAM (BRAM), memory able to store data
    - ✓ Can store a fair amount of data but slow and with a limited number of

# Register Transfer Level (RTL)

**Register Transfer Level (RTL)** is a hardware design abstraction that describes a **digital system in terms of data transfers between registers and the operations performed on that data in each clock cycle.**

At the RTL level, a design is defined by:

- **Registers** (storage elements)
- **Combinational logic** (operations on data)
- **Clocked behavior** (when data moves)

# ASIC (Application-Specific Integrated Circuit)

An **ASIC** is an **integrated circuit designed for a specific application or function**, rather than for general-purpose use.

## Key Characteristics

- **Application-specific:** Designed for one defined function
- **High performance:** Faster than general-purpose processors
- **Low power consumption:** Optimized for energy efficiency
- **Small area:** Minimal hardware overhead
- **High initial cost:** Design and fabrication are expensive
- **Low per-unit cost:** Cost-effective in high-volume production

## Examples of ASIC Applications

- Smartphone processors (baseband, image signal processor)
- Network routers and switches
- Cryptographic accelerators
- Automotive control units
- AI/ML accelerators
- Embedded controllers in consumer electronics

# VHDL- Very High Speed Integrated Circuit Hardware Description Language

- **VHDL is a hardware description language (HDL) used to model, simulate, and synthesize digital hardware such as FPGAs and ASICs.**
- **Unlike software languages, VHDL describes hardware structure and behavior that runs in parallel.**
- **VHDL is used for**
  - **Designing digital circuits** (combinational & sequential)
  - **RTL modeling** (Register Transfer Level)
  - **Simulation & verification** before hardware fabrication
  - **FPGA/ASIC synthesis**

# High Level Hardware Synthesis (HLS)

- High Level Synthesis (HLS) is an **Electronic Design Automation (EDA)** process that converts an **algorithm described in a high-level programming language**, such as **C, C++, or SystemC**, into a **Register Transfer Level (RTL)** hardware description, typically written in **VHDL or Verilog**.
- HLS bridges the gap between **software-level algorithm design** and **hardware-level implementation** by automatically generating optimized digital circuits from high-level code.

# High Level Hardware Synthesis (HLS)

HLS allows designers to describe hardware using:

- Algorithmic descriptions
- Behavioral models
- High-level languages (C, C++, SystemC, MATLAB)

and then generate optimized hardware architectures.

# Mapping Specification to Hardware

In HLS, the system specification is **mapped to hardware architecture** through several synthesis steps.

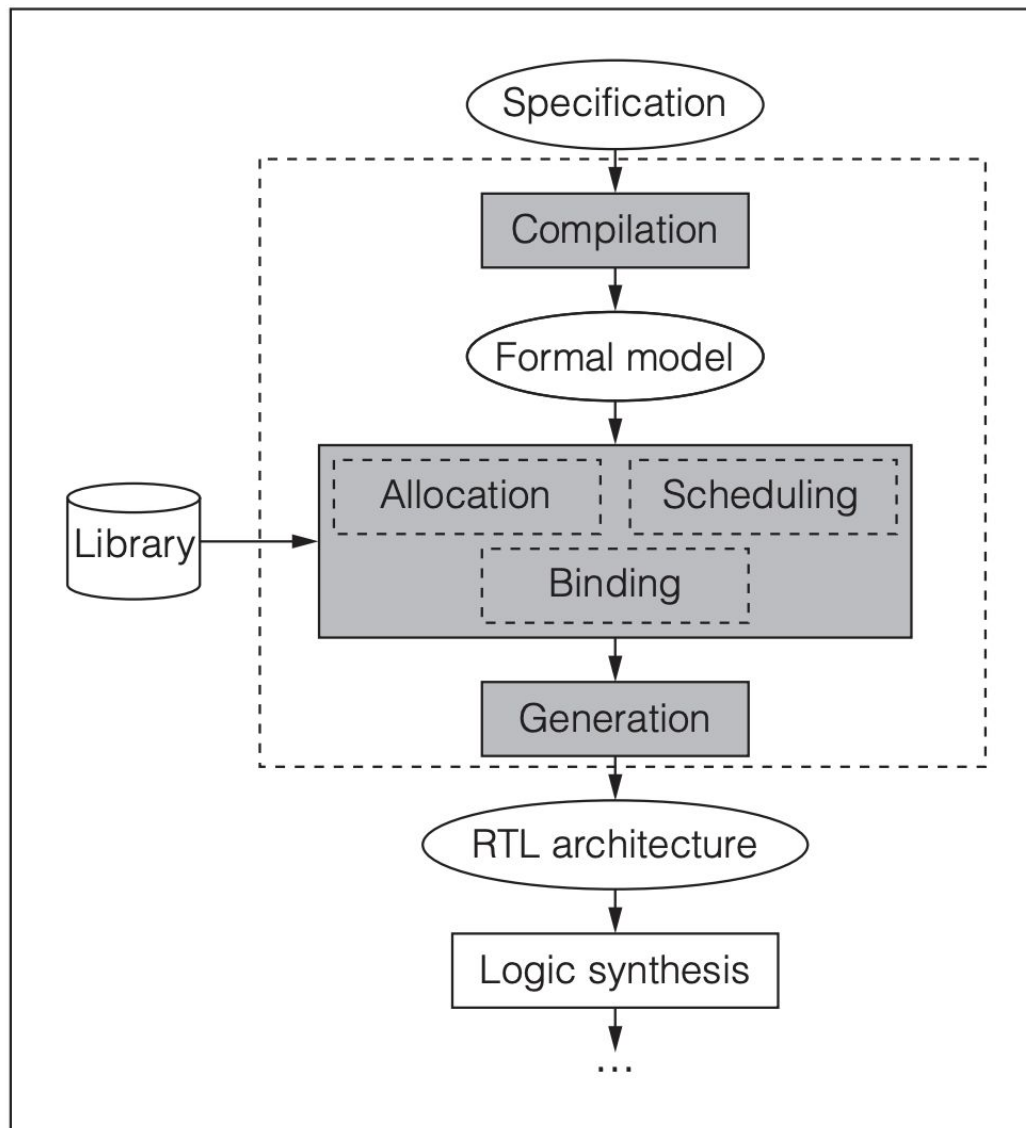
This process is especially important for **data-dominated systems**, such as:

- Digital Signal Processing (DSP)
- Image and video processing
- Machine learning accelerators
- IoT data analytics engines

The goal is to determine:

- Hardware components
- Execution order
- Resource usage
- Timing behavior

# HLS Design Flow



**Figure 1.** High-level synthesis (HLS) design steps.

# HLS Design Steps

## Specification

- The design process starts with a **high-level functional description** of the system (e.g., C, C++, SystemC).
- This description focuses on *what the system should do*, not how it is implemented in hardware.

## Compilation

- The high-level specification is compiled into an internal representation.
- Code optimizations and dependency analysis are performed.

## Formal Model

- The compiled design is converted into a **formal model** (such as DFG or CDFG).
- This model explicitly represents **data and control dependencies** between operations.

# HLS Design Steps

## Allocation

- Determines **what hardware resources** are needed (ALUs, multipliers, registers, buses).
- Resources are selected from a **library** containing hardware components with known area, delay, and power.

## Scheduling

- Decides **when each operation executes**, i.e., assigns operations to clock cycles.
- Ensures data dependencies and timing constraints are satisfied.

## Binding

- Assigns operations to specific functional units and variables to storage elements.
- Enables **resource sharing** and hardware optimization.

# HLS Design Steps

## Generation

- Based on allocation, scheduling, and binding decisions, an **RTL description** is generated.

## RTL Architecture

- The output consists of:
  - **Datapath** (registers, functional units, interconnects)
  - **Controller** (finite state machine)

## Logic Synthesis

- The RTL design is passed to logic synthesis tools.
- It is converted into a **gate-level implementation** for FPGA or ASIC.

# High Level Hardware Synthesis (HLS) in Embedded Systems

## Why High Level Hardware Synthesis (HLS) is Used in Embedded Systems

High Level Hardware Synthesis (HLS) is widely used in **embedded systems** because it helps designers to manage **complexity, performance, and time-to-market** while targeting **resource-constrained hardware** such as FPGAs and ASICs.

In short, High Level Hardware Synthesis is used in embedded systems to reduce design complexity and development time while generating optimized, high-performance, and low-power hardware from high-level algorithmic descriptions.

# Data Flow Graph

A *Data Flow Graph*,  $DFG = \{O \cup V, D\}$ , of a system is a directed graph with:

- $O$  = set of operations of the system (add, multiply, subtract, etc.),
- $V$  = set of operands/variables of the system and
- $D$  = set of edges, data dependencies between operands and operators.

An edge  $d = (o1, v1)$  or  $d = (v1, o1)$  means: Operation  $o1$  produces a value for  $v1$ , resp.  $v1$  is operand of  $o1$ .

# System Specification (Differential Equation)

## System Specification:

The system to be implemented in hardware solves the following differential equation numerically:

$$\frac{d^2y}{dx^2} + 5\frac{dy}{dx}x + 3y = 0$$

This equation is:

- Continuous (mathematical)
- Needs to be converted into a **discrete numerical algorithm** before hardware implementation

# Algorithm Specification

## Example: Algorithm spec in VHDL

```
...
loop
  x1 := x + dx;
  y1 := y + (u * dx);
  u := u - (5 * x * u * dx) - (3 * y * dx);
  x := x1;
  y := y1;
  exit when not (x1 < a);
end loop
...
```

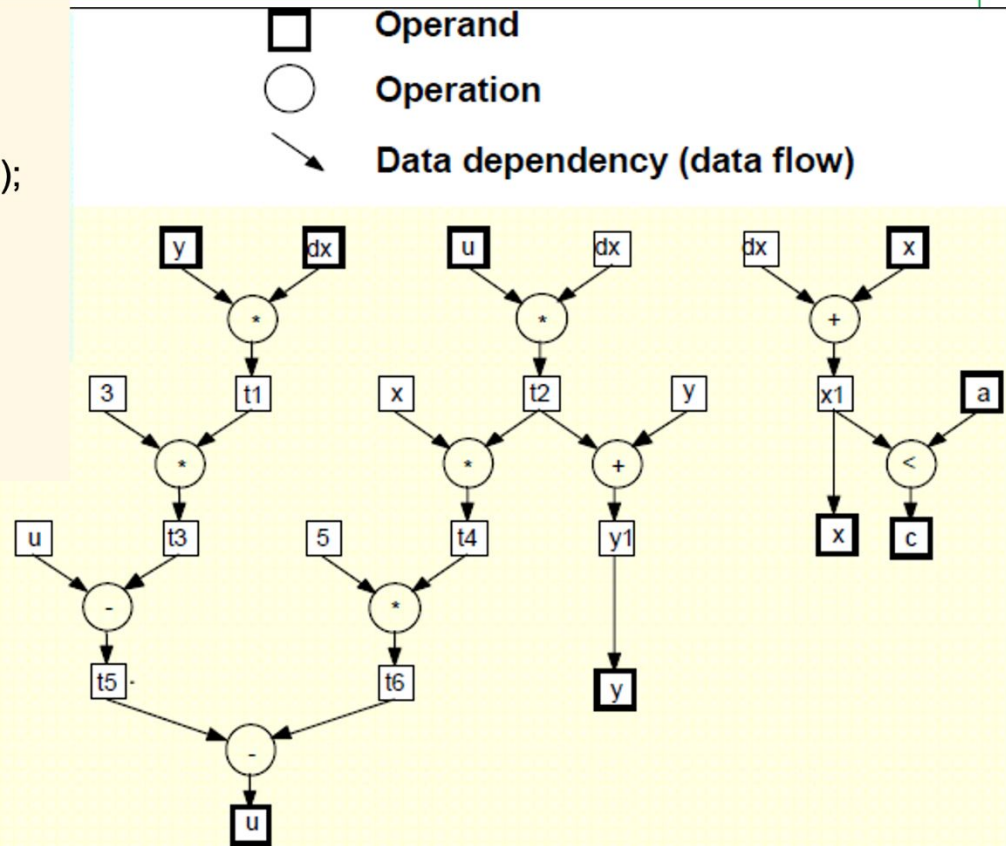
### Meaning of Variables

- $x \rightarrow$  Independent variable
  - $y \rightarrow$  Function value
  - $u \rightarrow$  First derivative ( $dy/dx$ )
  - $dx \rightarrow$  Step size
  - $x1, y1 \rightarrow$  Next iteration values
-  This loop represents **one iteration** of numerical integration.

- ✓ The primary goal shown in the VHDL specification is to achieve **maximum parallelization**.
- ✓ In hardware design (like FPGAs or ASICs), unlike standard software that runs line-by-line, multiple operations can occur simultaneously if they do not depend on each other's immediate results.

# DFG of the Differential Equation Solver

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) - (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



# Scheduling

- ✓ **Scheduling** is the process of deciding exactly when each operation in a Data Flow Graph (DFG) will be executed.
- ✓ It acts as a bridge between a high-level algorithm and the physical hardware implementation.

**Goal:** Assign the execution of operations to specific (clock) cycles.

**Approaches:**

- ✓ **Scheduling without constraints: ASAP, ALAP**
  - **Result:** fastest possible implementation (at no cost limits)
- ✓ **Resource constrained scheduling**
  - **Constraint:** fixed maximum amount of parallel operations (cost)
  - **Optimization:** length of the schedule is minimized (max. performance)
- ✓ **Time constrained scheduling**
  - **Constraint:** fixed maximum length of the schedule (performance)
  - **Optimization:** amount of parallel operations (cost) is minimized

# **Basic Scheduling Algorithms: ASAP, ALAP**

# ASAP Scheduling (As Soon As Possible)

## Definition:

**ASAP scheduling** assigns each operation to the **earliest possible clock cycle**, as soon as all its input data is available.

## Key Idea:

Execute operations **as early as dependencies allow**.

## Working of ASAP Scheduling:

Given the algorithm:

$x1 := x + dx;$

$y1 := y + (u * dx);$

$u := u - (5 * x * u * dx) - (3 * y * dx);$

$x := x1;$

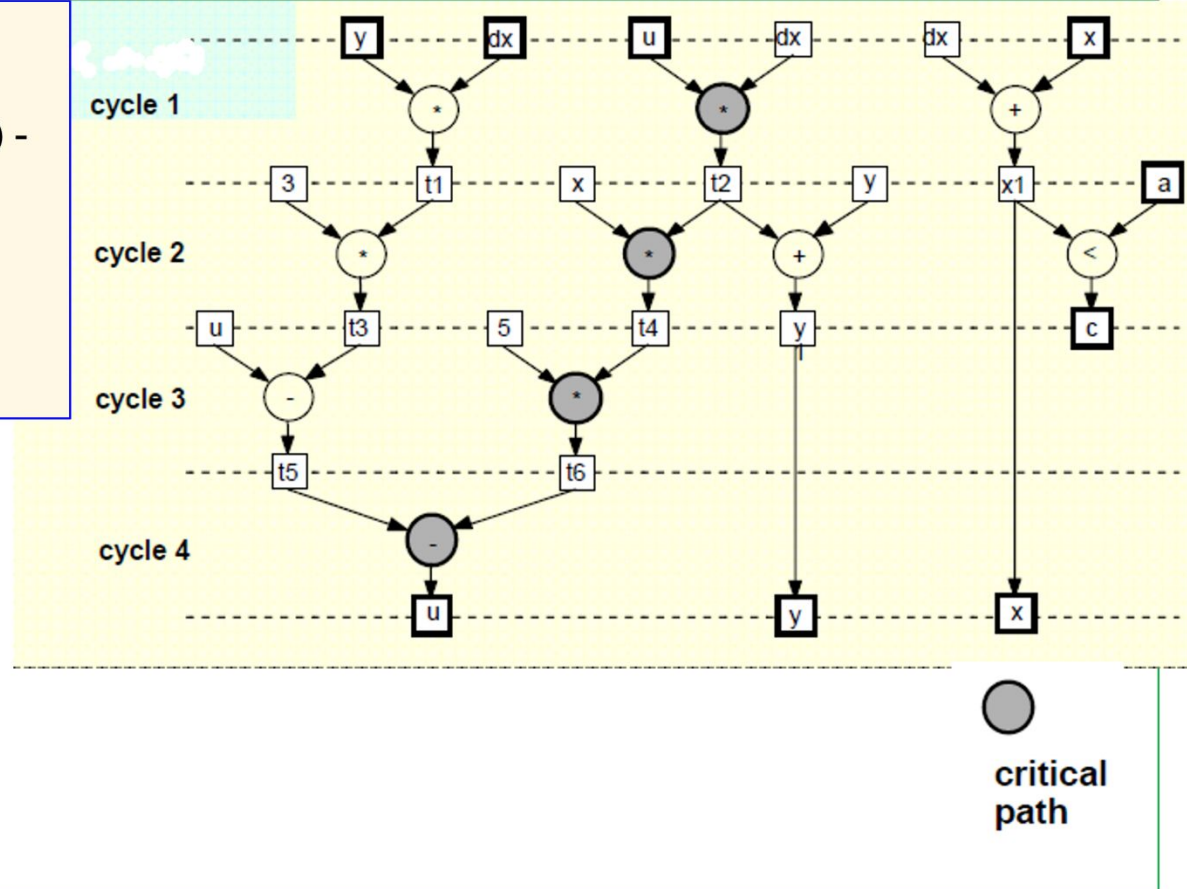
$y := y1;$

## Steps:

- Start from **input nodes**
- Schedule operations whose operands are ready
- Move forward **cycle by cycle**

# Example: ASAP Scheduling

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) -  
  (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



## Cycle 1

Operations whose inputs are immediately available:

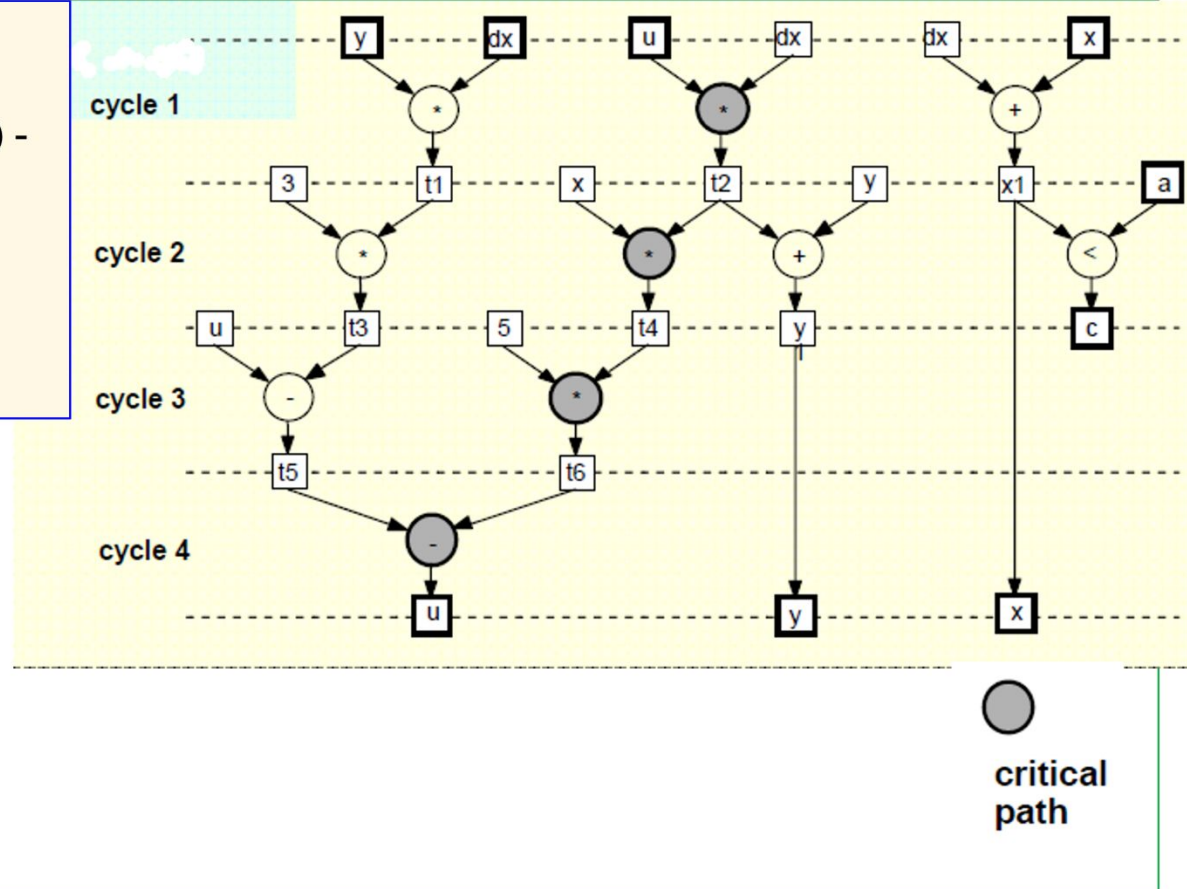
- $x1 = x + dx \rightarrow$  addition
- $u * dx \rightarrow$  multiplication
- $y * dx \rightarrow$  multiplication
- $y + (u * dx)$  **cannot finish yet** (depends on  $u * dx$ )



Independent operations start **in parallel**.

# Example: ASAP Scheduling

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) -  
  (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



## Cycle 2

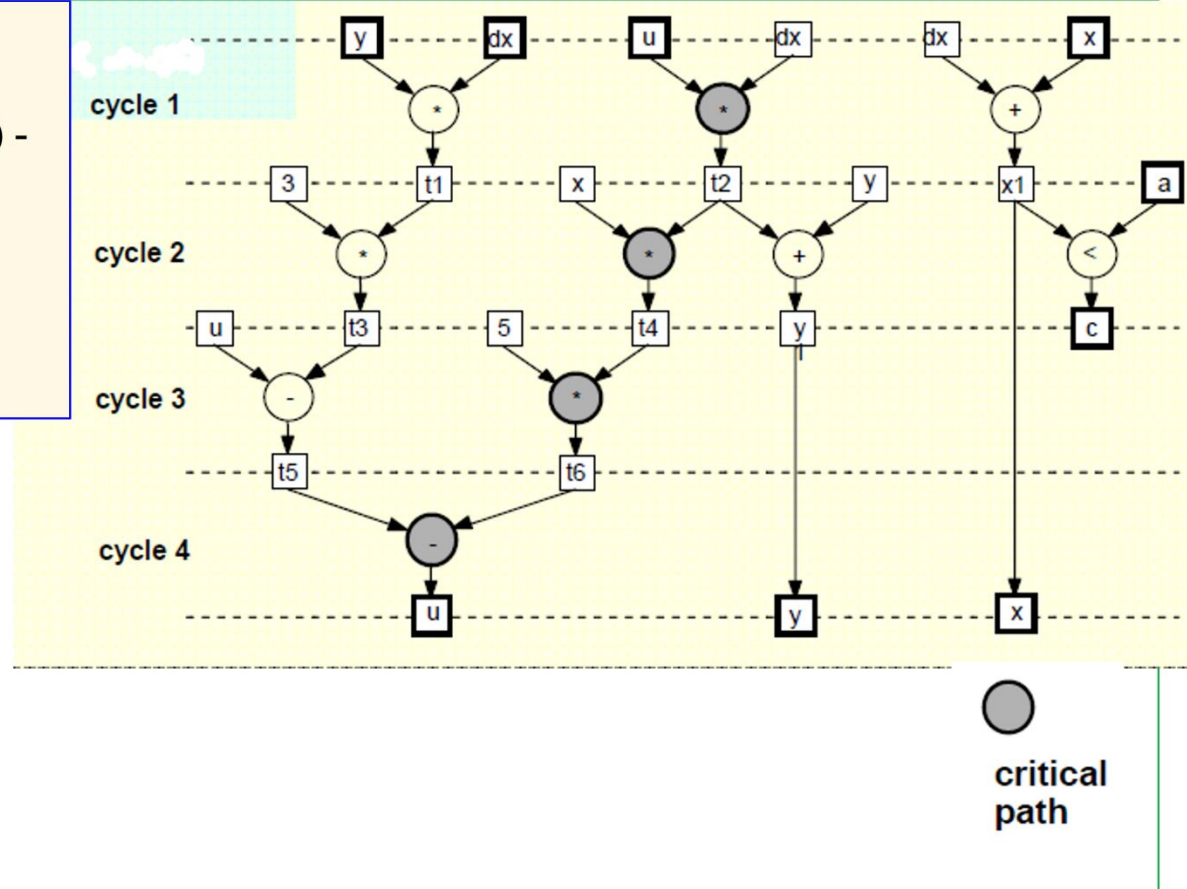
Now results from cycle 1 are available:

- $y1 = y + (u * dx) \rightarrow$  addition
- $x * (u * dx) \rightarrow$  multiplication
- Partial computations for  $5 * x * u * dx$  and  $3 * y * dx$  begin

👉 More parallel arithmetic is exposed.

# Example: ASAP Scheduling

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) -  
  (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



## Cycle 3

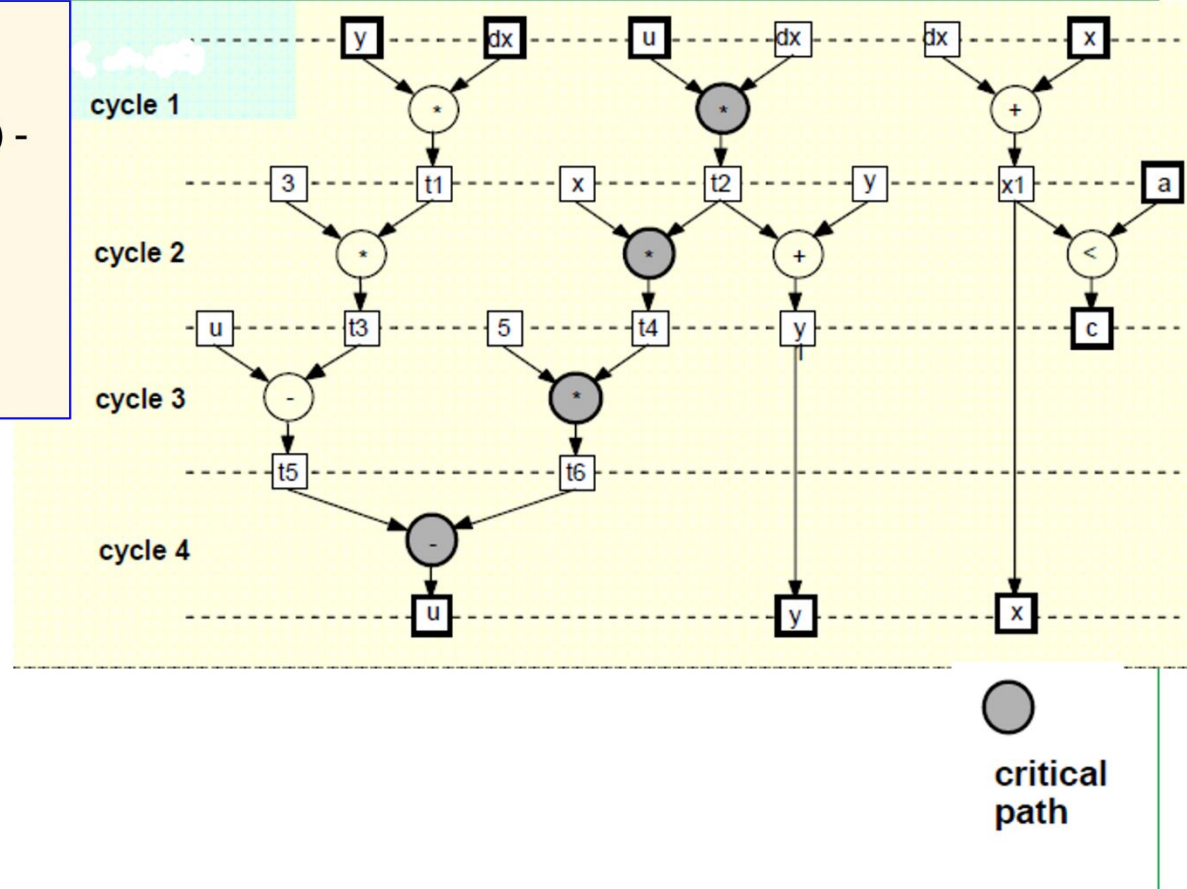
Dependent arithmetic continues:

- $5 * (x * u * dx)$
- $3 * y * dx$
- Intermediate subtraction steps start forming

These operations **depend on earlier multiplications**, so they cannot start earlier.

# Example: ASAP Scheduling

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) -  
  (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



## Cycle 4

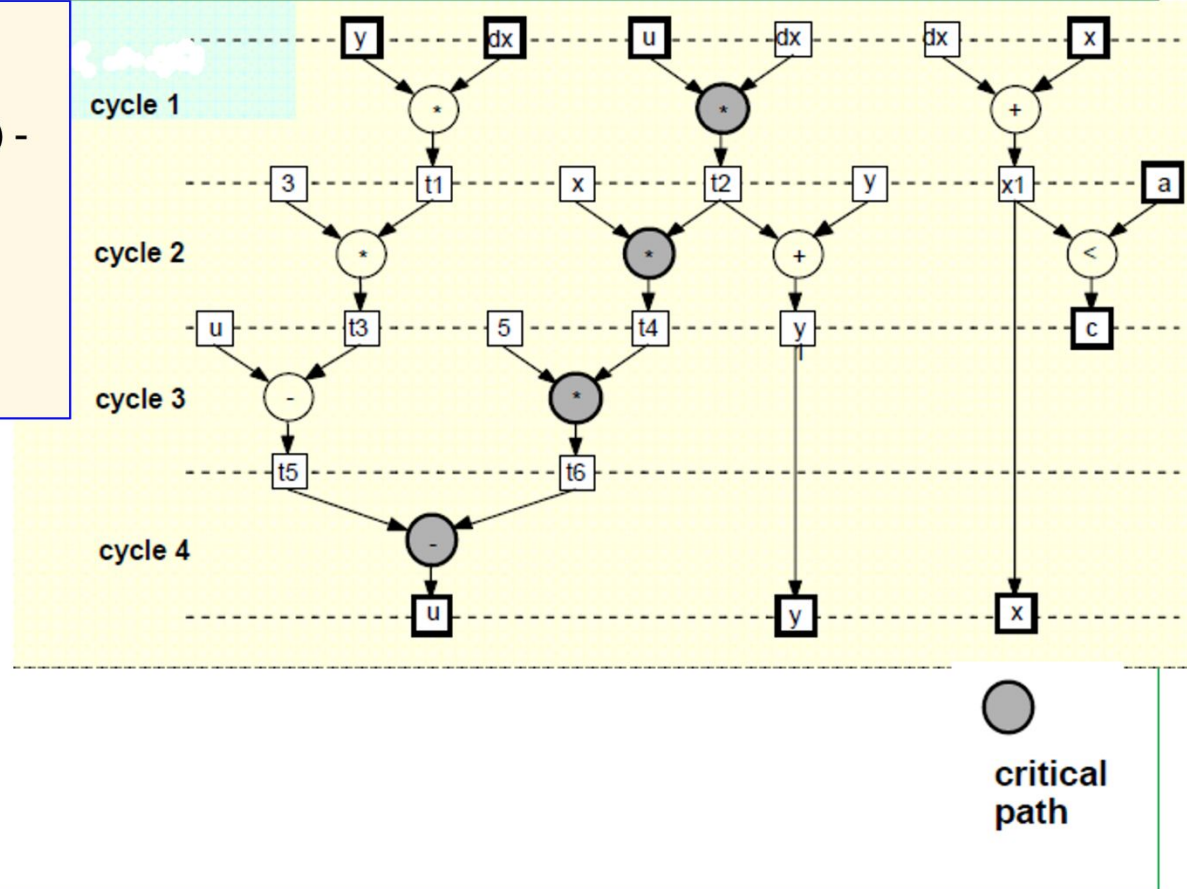
Final dependent operations complete:

- $u = u - (5 * x * u * dx) - (3 * y * dx)$
- Register updates:
  - $x := x1$
  - $y := y1$

At this point, the **iteration completes**.

# Example: ASAP Scheduling

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) -  
  (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



## Critical Path (Gray Nodes)

- The **critical path** is the **longest chain of dependent operations**
- It determines the **minimum number of cycles (latency)** → here, 4 cycles
- Operations on this path **cannot be moved earlier**

👉 ASAP scheduling highlights this path clearly.

# ALAP Scheduling (As Late As Possible)

- **ALAP scheduling** assigns each operation to the **latest possible clock cycle without increasing the total execution time (latency)**.
- The **final schedule length is fixed** (taken from ASAP or given by a time constraint).

# Working of ALAP Scheduling

## Step 1: Fix the Final Cycle

- The last operations must finish in **cycle 4**
- This is determined by the **critical path**

## Step 2: Traverse the DFG Backwards

- Start from **output nodes** (x, y, u)
- Move **backward** toward inputs (x, y, u, dx)
- Place each operation **as late as possible**, while:
  - Respecting data dependencies
  - Not exceeding the fixed schedule length

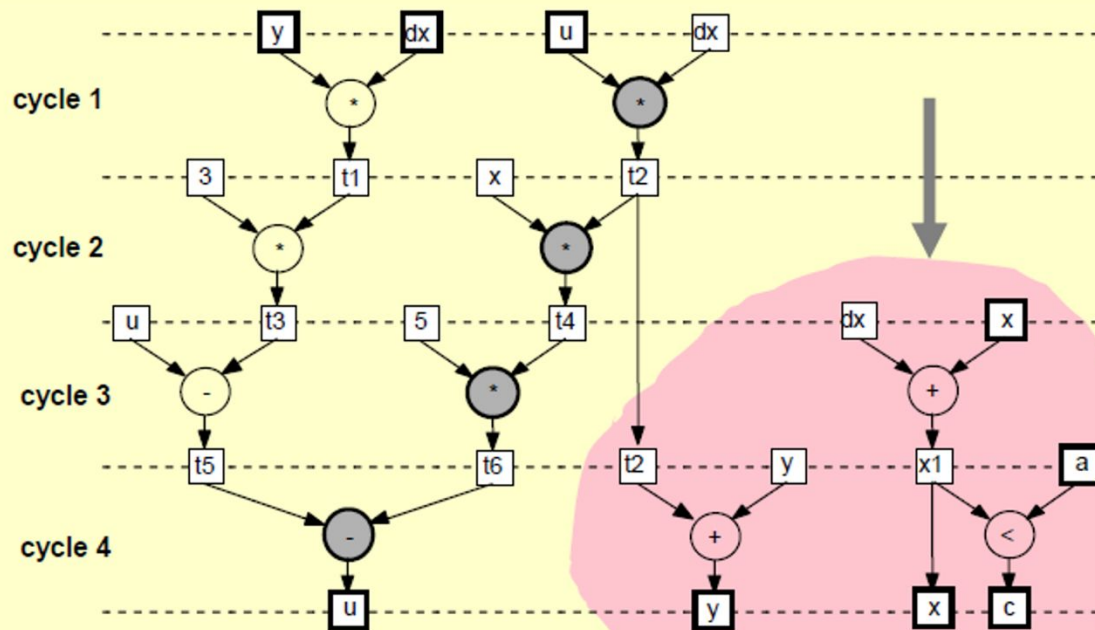
# Example: ALAP Scheduling

## ALAP-Scheduling

- ▶ ALAP = As Late As Possible
- ▶ The data flow graph is processed backwards, from the output operands towards the input operands.

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) - (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```

 critical path



# Topics

- Mapping Specification → HW by High Level Synthesis / Design (architecture synthesis for data-dominated systems)
  - High Level Synthesis Overview
  - Basic Scheduling Algorithms: ASAP, ALAP
  - **Resource-Constrained Scheduling**
    - Time-Constrained Scheduling
    - Advanced Scheduling: Operator Pipelining, Chaining & Multicycling
    - Allocation, Binding, Data Path & Controller Generation
    - Pipelined Data Path Design
    - Pipeline Controller Design

# Resource-Constrained Scheduling

**Resource Constrained Scheduling** is a scheduling technique in **High Level Synthesis (HLS)** where:

- The **number of available hardware resources** (adders, multipliers, etc.) is **limited**
- Operations must be scheduled **within these limits**
- The objective is to **finish execution in the minimum number of clock cycles**

 Unlike ASAP scheduling, **not all parallel operations can be executed at once.**

 **Goal:** Scheduling with minimal number of cycles under consideration of given constraints on available resources of each type.

# Resource-Constrained Scheduling

## Approach: List Scheduling

- ❖ Operations whose input operands are available are inserted into a list according to their priority (rather than scheduling them immediately, **ASAP**). The priority is computed according to a heuristics.
- ❖ In each cycle the **highest priority** operations are scheduled until all resources are used or the operations list is empty.
- ❖ Due to this, new operations become executable and the previous steps are repeated until the complete data flow graph has been scheduled.

# Resource-Constrained Scheduling

## List Scheduling

### Step: Create a Ready List

- ❖ Operations whose **input operands are available**
- ❖ These operations are **not executed immediately**
- ❖ Instead, they are placed in a **list**

## Priority-Based Scheduling

- ❖ Each operation in the list is assigned a **priority**
- ❖ Priority is decided using **heuristics**, such as:
  - ❖ Critical path length
  - ❖ Mobility (ALAP – ASAP)
  - ❖ Operation type (e.g., multiplier first)

 Operations with **higher priority** are scheduled first.

# Resource-Constrained Scheduling

## Scheduling Per Clock Cycle

### In each clock cycle:

- ❖ Pick the **highest-priority operations**
- ❖ Schedule them **until all resources are used**
- ❖ If resources are exhausted → remaining operations wait

### Example:

- ❖ 1 multiplier available
- ❖ 3 multiplications ready
- ❖ Only **1 multiplication** can be scheduled this cycle

# Resource-Constrained Scheduling

## Iterative Process

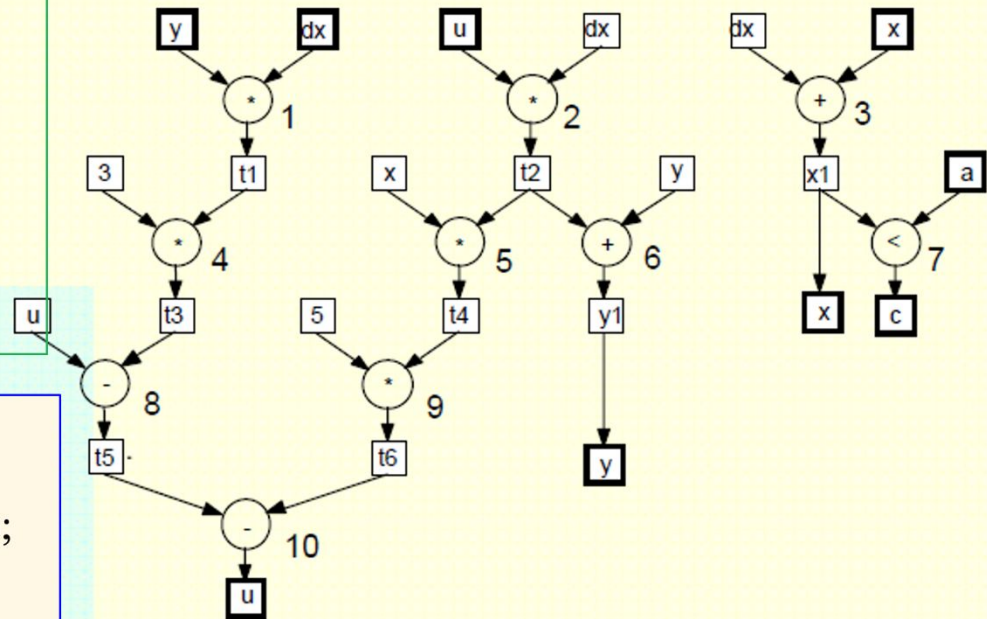
### After scheduling a cycle:

- ❖ Some operations finish
  - ❖ Their results become available
  - ❖ **New operations become ready**
  - ❖ These are added to the list
  - ❖ The process repeats
- ✓ Continues until the **entire Data Flow Graph (DFG)** is scheduled.

# List Scheduling

- ▶ Resource constraints:
- ▶ 2 Multiplier
- ▶ 1 Adder
- ▶ 1 subtractor
- ▶ 1 ALU (<)

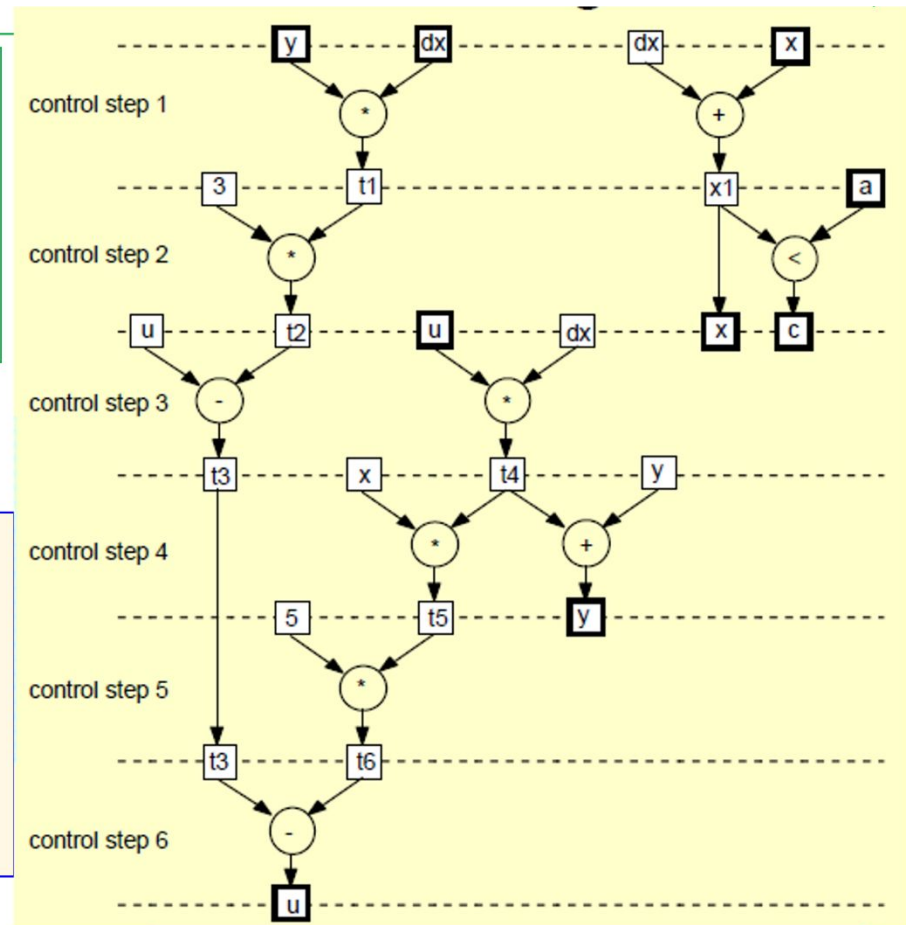
```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) - (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



# List Scheduling

- ▶ Resource constraints:
- ▶ 1 Multiplier
- ▶ 1 ALU (+, -, <)

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u * dx) - (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```



# Topics

- Mapping Specification → HW by High Level Synthesis / Design (architecture synthesis for data-dominated systems)
  - High Level Synthesis Overview
  - Basic Scheduling Algorithms: ASAP, ALAP
  - Resource-Constrained Scheduling
  - **Time-Constrained Scheduling**
    - Advanced Scheduling: Operator Pipelining, Chaining & Multicycling
    - Allocation, Binding, Data Path & Controller Generation
    - Pipelined Data Path Design
    - Pipeline Controller Design

# Time-Constrained Scheduling

**Time Constrained Scheduling** is a scheduling method in **High Level Synthesis (HLS)** where:

- ❖ The **total number of clock cycles (latency)** is **fixed**
- ❖ The goal is to use the **minimum number of hardware resources**
- ❖ Hardware cost (area, power) is minimized while meeting timing constraints

 This is the **opposite objective** of resource-constrained scheduling.

# Time-Constrained Scheduling

 **Goal:** Scheduling with **minimal number of resources** under consideration of given **constraints on available time** (number of cycles).

- ❖ Execution time (number of cycles) is **given**
- ❖ We must fit all operations **within this deadline**
- ❖ Use **as few adders, multipliers, comparators, etc. as possible**

# Time-Constrained Scheduling

## Approach: Force-Directed Scheduling

- ❖ **Basic idea:** the degree of concurrency of operations must be balanced over all cycles (control steps).
- ❖ ASAP and ALAP schedules are computed for the given number of cycles to derive the time frame in which each operation can be scheduled.
- ❖ For each type of operation, a **distribution graph (DG)** is built to denote the possible control steps for that operation. If an operation can be done in  $k$  steps, then its part in the DG is weighted with  $1/k$ .
- ❖ A force is defined that decreases when the DG is more balanced.
- ❖ Operations are assigned to cycles so that the force is minimized.

# Time-Constrained Scheduling

## Role of ASAP and ALAP

- ❖ **ASAP schedule** → earliest possible execution time
- ❖ **ALAP schedule** → latest possible execution time

From these:

- ❖ A **time frame (Mobility Range)** is defined for each operation.
- ❖ **Mobility / Movability range:** the set of cycles where an operation *may* be placed.

$$\text{Mobility range} = [t_{ASAP}, t_{ALAP}]$$

 This time frame tells **where an operation is allowed to be scheduled**.

# Time-Constrained Scheduling

## Distribution Graph (DG)

Not a DFG (no vertices/edges). DG is a **cycle-wise distribution** showing “how much of an operation type is expected/allowed in each cycle” based on mobility.

- ❖ Built for each type of operation (add, multiply, etc.)
- ❖ Shows **probability distribution** of operations across cycles

## Weighting Rule

If an operation can be scheduled in **k cycles**:

- ❖ It contributes  **$1/k$**  to each of those cycles



This models **resource usage probability**.

# Time-Constrained Scheduling

## Force Concept

- ❖ **Force** measures how unbalanced the distribution graph is
- ❖ High force → many operations concentrated in few cycles
- ❖ Low force → operations evenly distributed

 A balanced DG means **fewer resources needed**

# Time-Constrained Scheduling

## Scheduling Decision

- ❖ Operations are assigned to specific cycles
- ❖ Assignment chosen such that **overall force is minimized**
- ❖ This reduces peak parallelism

 Result: **minimum hardware resources** within fixed time.

# Time-Constrained Scheduling

## Scheduling Decision

- ❖ Operations are assigned to specific cycles
- ❖ Assignment chosen such that **overall force is minimized**
- ❖ This reduces peak parallelism

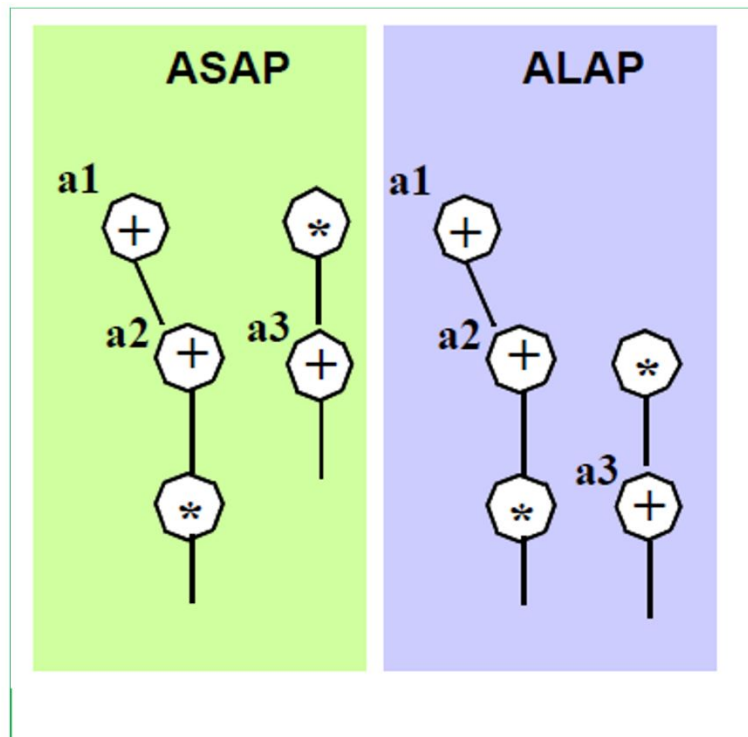
 Result: **minimum hardware resources** within fixed time.

# Example: Force Directed Scheduling

Given an algorithmic specification

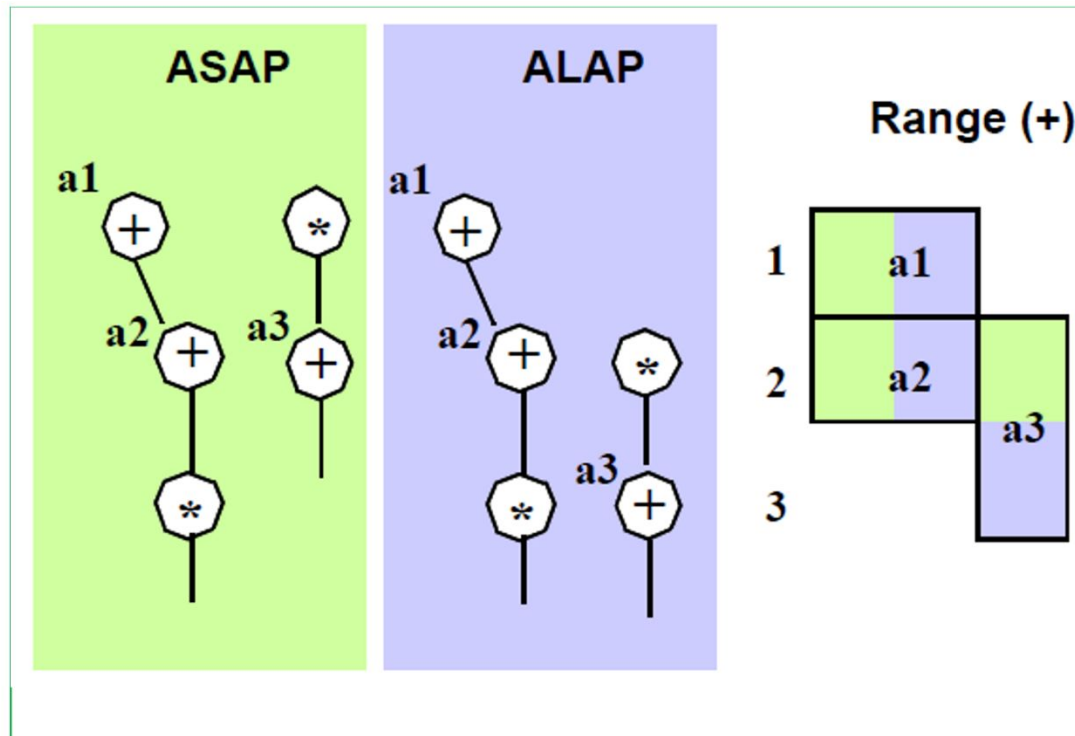
# Example: Distribution Graph

**Step-01:** Construct the ASAP and ALAP schedules.



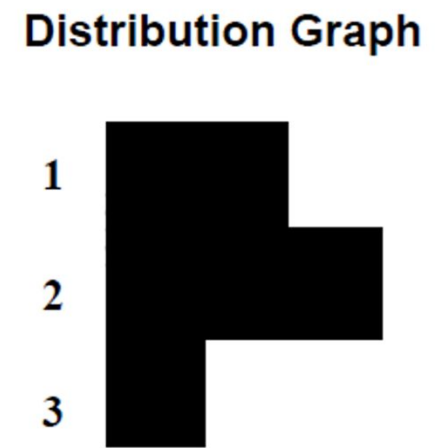
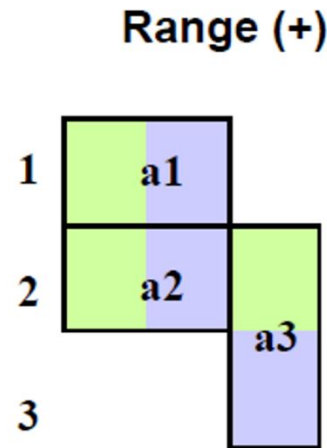
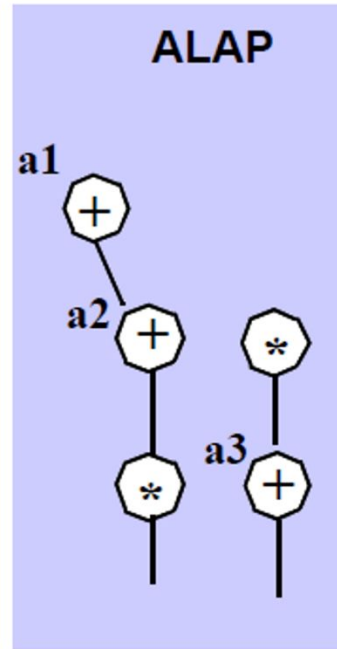
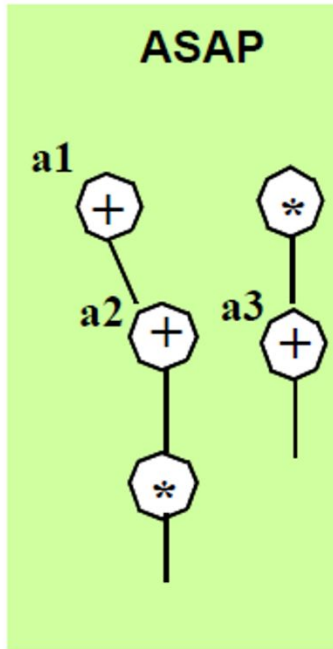
# Example: Distribution Graph

**Step-02:** Calculate the Mobility / Movability range from the DFG of ASAP and ALAP scheduling.



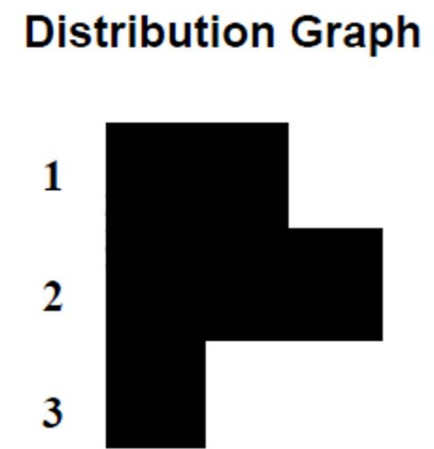
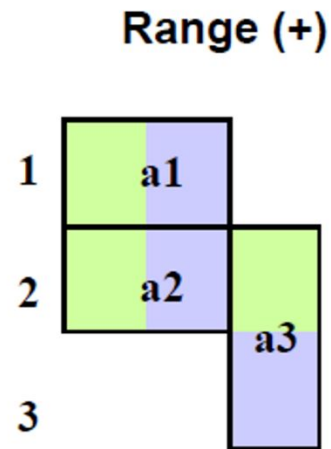
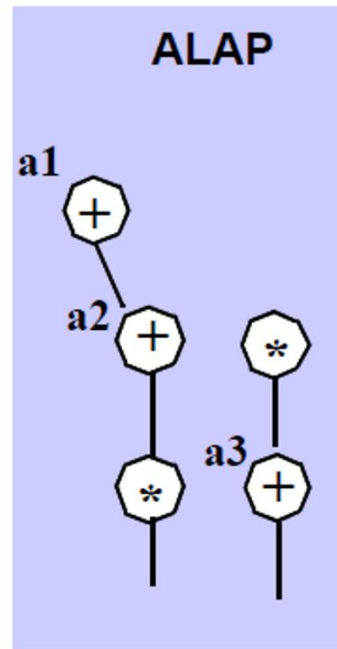
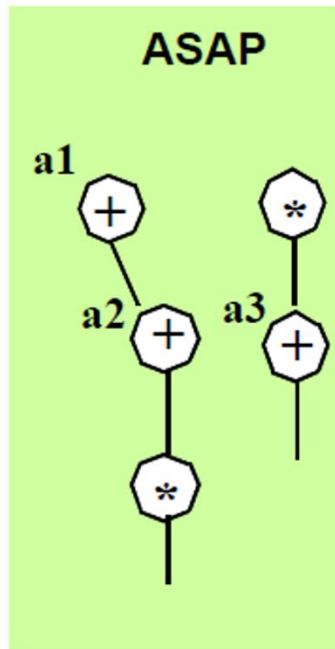
# Example: Distribution Graph

**Step-03:** Construct the Distribution Graph (DG).



# Example: Distribution Graph

**Step-:** Construct the ASAP and ALAP schedules.



# Example: Time-Constrained Scheduling

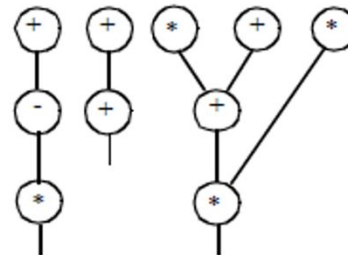
## Algorithm:

```
a := i1 + i2;  
o1 := (a - i3) * 3;  
o2 := i4 + i5 + i6;  
d := i7 * i8;  
g := d + i9 + i10;  
o3 := i11 * 7 * g;
```

## Time Constraint:

4 cycles (control steps)

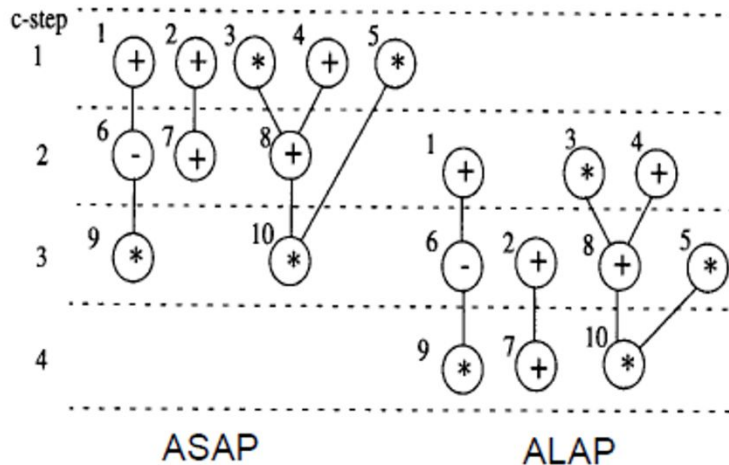
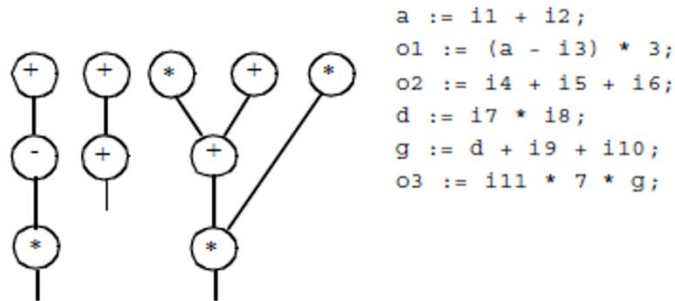
**Question:** Is this feasible?



```
a := i1 + i2;  
o1 := (a - i3) * 3;  
o2 := i4 + i5 + i6;  
d := i7 * i8;  
g := d + i9 + i10;  
o3 := i11 * 7 * g;
```

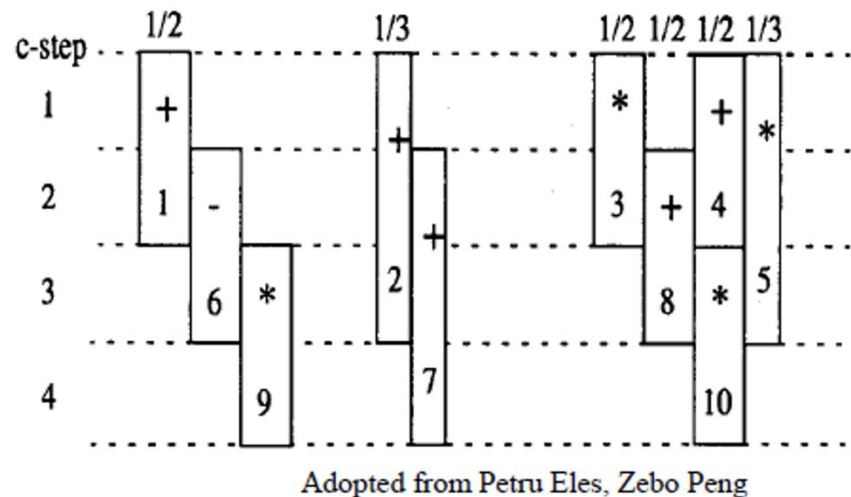
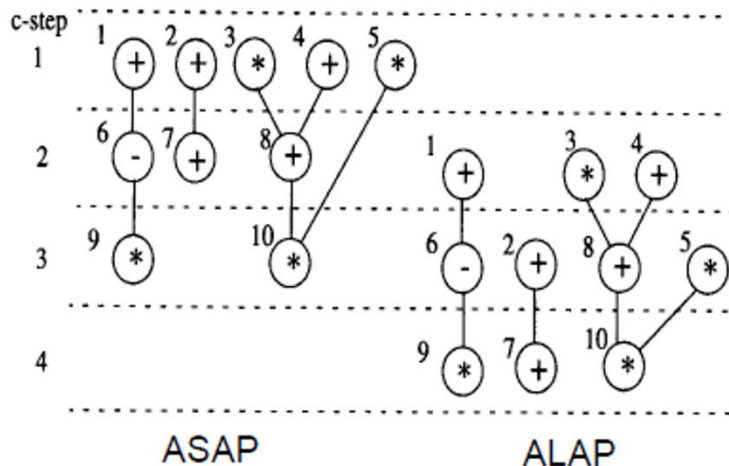
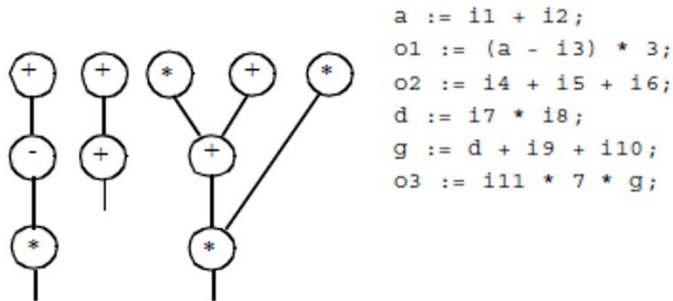
## Example: Time-Constrained Scheduling

## Step-01: Construct the ASAP and ALAP schedules.



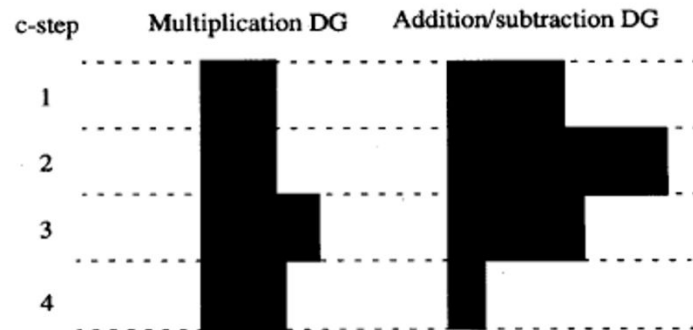
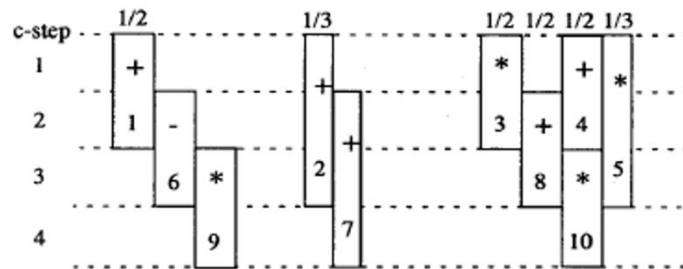
# Example: Time-Constrained Scheduling

**Step-02:** Calculate the Mobility / Movability range from the DFG of ASAP and ALAP scheduling.



# Example: Time-Constrained Scheduling

## Step-03: Construct the Distribution Graph (DG).



| Res. | c-step | Calc.                     | DG    |
|------|--------|---------------------------|-------|
| Mult | 1      | $1/2+1/3$                 | 0.833 |
| Mult | 2      | $1/2+1/3$                 | 0.833 |
| Mult | 3      | $1/2+1/2+1/3$             | 1.333 |
| Mult | 4      | $1/2+1/2$                 | 1.0   |
| +/-  | 1      | $1/2+1/3+1/2$             | 1.333 |
| +/-  | 2      | $1/2+1/2+1/3+1/3+1/2+1/2$ | 2.667 |
| +/-  | 3      | $1/2+1/3+1/3+1/2$         | 1.667 |
| +/-  | 4      | $1/3$                     | 0.333 |

# Example: Time-Constrained Scheduling

## Step-04: Force calculation.

The algorithm tries to balance the distribution graph by calculating the force of each **operation-to-control** step assignment and selecting the smallest force.

*With*

$o_i$  operation  $i$ ,

$s_j$  control step  $j$ ,

$\Delta T(o_i)$  number of possible control steps for  $o_i$ ,

$DG(s)$  distribution graph for  $s$ :

$$\text{Force}_{(\sigma(o_i) = s_j)} = DG(s_j) - \frac{1}{\Delta T(o_i)} \cdot \sum_{s = \sigma_{ASAP}(o_i)}^{\sigma_{ALAP}(o_i)} DG(s)$$

# Example: Time-Constrained Scheduling

- ❖ The force is calculated for each operation  $o_i$  and each cycle (control step)  $s_j$  in which the operation can be scheduled.
- ❖ The calculation is by subtracting the average of the distribution graph over all cycles in which the operation can be scheduled (to the right of the  $-$ ) and subtracting it from the value of the distribution graph for the cycle  $s_j$ .
- ❖ Note that the distribution graph for the resource type which implements the operation has to be used.

$$\text{Force}_{(\sigma(o_i) = s_j)} = \text{DG}(s_j) - \frac{1}{\Delta T(o_i)} \cdot \sum_{s = \sigma_{\text{ASAP}}(o_i)}^{\sigma_{\text{ALAP}}(o_i)} \text{DG}(s)$$

- ❖ *The computation can be done systematically by setting up a table whose rows correspond to the different cycles (limited by the time constraint) and the columns correspond to the different operations. Make entries in the table cells if and only if the operation (given by the column) can be scheduled in the cycle (given by the row).*
- ❖ The smaller the force (note that forces may be negative!), the more contributes the scheduling of operation  $o_i$  in cycle  $s_j$  to a balanced DG.

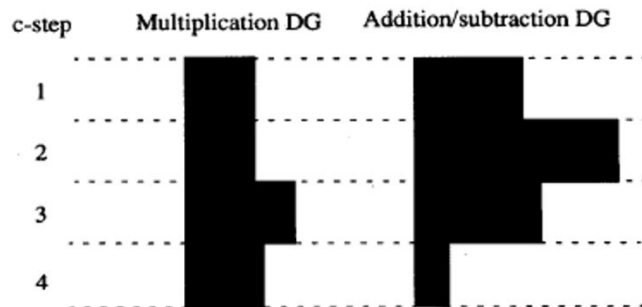
# Example: Time-Constrained Scheduling

## Step-04: Force calculation.

$$\text{Force}_{(\sigma(o_i) = s_j)} = \text{DG}(s_j) - \frac{1}{\Delta T(o_i)} \cdot \sum_{s = \sigma_{\text{ASAP}}(o_i)}^{\sigma_{\text{ALAP}}(o_i)} \text{DG}(s)$$

$$\begin{aligned} \text{Force}(3) &= \text{DG}_{\text{mult}}(3) - \text{avg. DG}_{\text{mult}}(\text{op10}) \\ &= 1.333 - (1.333 + 1.0)/2 \\ &= 0.167 \end{aligned}$$

$$\begin{aligned} \text{Force}(4) &= \text{DG}_{\text{mult}}(4) - \text{avg. DG}_{\text{mult}}(\text{op10}) \\ &= 1.0 - (1.333 + 1.0)/2 \\ &= -0.167 \end{aligned}$$



| Op   | c-step | Calc.                   | DG    |
|------|--------|-------------------------|-------|
| Mult | 1      | 1/2+1/3                 | 0.833 |
| Mult | 2      | 1/2+1/3                 | 0.833 |
| Mult | 3      | 1/2+1/2+1/3             | 1.333 |
| Mult | 4      | 1/2+1/2                 | 1.0   |
| +/-  | 1      | 1/2+1/3+1/2             | 1.333 |
| +/-  | 2      | 1/2+1/2+1/3+1/3+1/2+1/2 | 2.667 |
| +/-  | 3      | 1/2+1/3+1/3+1/2         | 1.667 |
| +/-  | 4      | 1/3                     | 0.333 |

Adopted from Petru Eles, Zebo Peng

### Explanation:

The example considers a **multiplication operation** labeled **op10**

From ASAP/ALAP analysis: **op10** can be scheduled in **cycle 3 or cycle 4**

Therefore:

$$\Delta T(\text{op10}) = 2$$

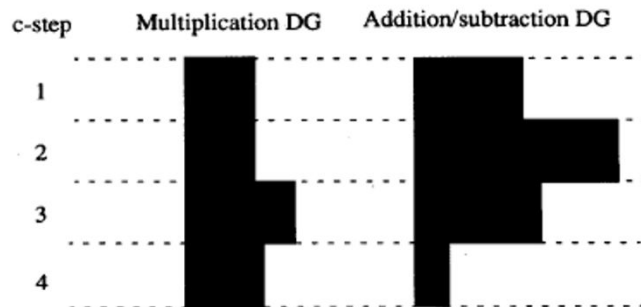
# Example: Time-Constrained Scheduling

## Step-04: Force calculation.

$$\text{Force}_{(\sigma(o_i) = s_j)} = DG(s_j) - \frac{1}{\Delta T(o_i)} \cdot \sum_{s = \sigma_{ASAP}(o_i)}^{\sigma_{ALAP}(o_i)} DG(s)$$

$$\begin{aligned} \text{Force}(3) &= DG_{\text{mult}}(3) - \text{avg. } DG_{\text{mult}}(\text{op10}) \\ &= 1.333 - (1.333 + 1.0)/2 \\ &= 0.167 \end{aligned}$$

$$\begin{aligned} \text{Force}(4) &= DG_{\text{mult}}(4) - \text{avg. } DG_{\text{mult}}(\text{op10}) \\ &= 1.0 - (1.333 + 1.0)/2 \\ &= -0.167 \end{aligned}$$



| Op   | c-step | Calc.                   | DG    |
|------|--------|-------------------------|-------|
| Mult | 1      | 1/2+1/3                 | 0.833 |
| Mult | 2      | 1/2+1/3                 | 0.833 |
| Mult | 3      | 1/2+1/2+1/3             | 1.333 |
| Mult | 4      | 1/2+1/2                 | 1.0   |
| +/-  | 1      | 1/2+1/3+1/2             | 1.333 |
| +/-  | 2      | 1/2+1/2+1/3+1/3+1/2+1/2 | 2.667 |
| +/-  | 3      | 1/2+1/3+1/3+1/2         | 1.667 |
| +/-  | 4      | 1/3                     | 0.333 |

Adopted from Petru Eles, Zebo Peng

Compute Average DG for op10:

$$\text{Average } DG_{\text{mult}}(\text{op10}) = \frac{DG_{\text{mult}}(3) + DG_{\text{mult}}(4)}{2} = \frac{1.333 + 1.0}{2} = 1.167$$

Since **op10** can be in **cycles 3 or 4**, only these are used.

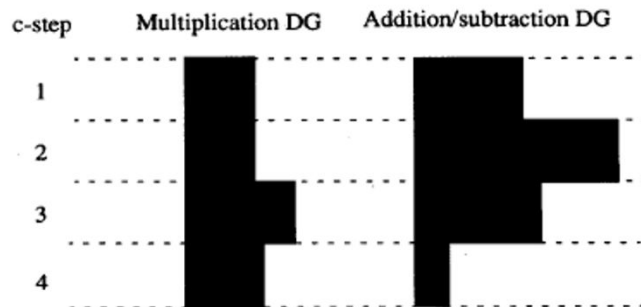
# Example: Time-Constrained Scheduling

## Step-04: Force calculation.

$$\text{Force}_{(\sigma(o_i) = s_j)} = DG(s_j) - \frac{1}{\Delta T(o_i)} \cdot \sum_{s = \sigma_{ASAP}(o_i)}^{\sigma_{ALAP}(o_i)} DG(s)$$

$$\begin{aligned} \text{Force}(3) &= DG_{\text{mult}}(3) - \text{avg. } DG_{\text{mult}}(\text{op10}) \\ &= 1.333 - (1.333 + 1.0)/2 \\ &= 0.167 \end{aligned}$$

$$\begin{aligned} \text{Force}(4) &= DG_{\text{mult}}(4) - \text{avg. } DG_{\text{mult}}(\text{op10}) \\ &= 1.0 - (1.333 + 1.0)/2 \\ &= -0.167 \end{aligned}$$



| Op   | c-step | Calc.                   | DG    |
|------|--------|-------------------------|-------|
| Mult | 1      | 1/2+1/3                 | 0.833 |
| Mult | 2      | 1/2+1/3                 | 0.833 |
| Mult | 3      | 1/2+1/2+1/3             | 1.333 |
| Mult | 4      | 1/2+1/2                 | 1.0   |
| +/-  | 1      | 1/2+1/3+1/2             | 1.333 |
| +/-  | 2      | 1/2+1/2+1/3+1/3+1/2+1/2 | 2.667 |
| +/-  | 3      | 1/2+1/3+1/3+1/2         | 1.667 |
| +/-  | 4      | 1/3                     | 0.333 |

Adopted from Petru Eles, Zebo Peng

Force if op10 is placed in cycle 3:

$$\text{Force}(3) = DG_{\text{mult}}(3) - \text{avg. } DG = 1.333 - 1.167 = \boxed{+0.167}$$

**Interpretation:**

Cycle 3 is already **heavily loaded**  
Placing op10 here **increases imbalance**

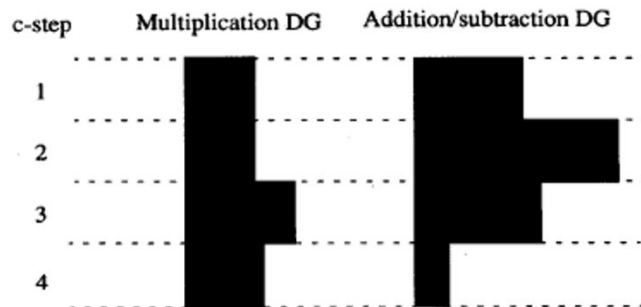
# Example: Time-Constrained Scheduling

## Step-04: Force calculation.

$$\text{Force}_{(\sigma(o_i) = s_j)} = DG(s_j) - \frac{1}{\Delta T(o_i)} \cdot \sum_{s = \sigma_{ASAP}(o_i)}^{\sigma_{ALAP}(o_i)} DG(s)$$

$$\begin{aligned} \text{Force}(3) &= DG_{\text{mult}}(3) - \text{avg. } DG_{\text{mult}}(\text{op10}) \\ &= 1.333 - (1.333 + 1.0)/2 \\ &= 0.167 \end{aligned}$$

$$\begin{aligned} \text{Force}(4) &= DG_{\text{mult}}(4) - \text{avg. } DG_{\text{mult}}(\text{op10}) \\ &= 1.0 - (1.333 + 1.0)/2 \\ &= -0.167 \end{aligned}$$



| Op   | c-step | Calc.                   | DG    |
|------|--------|-------------------------|-------|
| Mult | 1      | 1/2+1/3                 | 0.833 |
| Mult | 2      | 1/2+1/3                 | 0.833 |
| Mult | 3      | 1/2+1/2+1/3             | 1.333 |
| Mult | 4      | 1/2+1/2                 | 1.0   |
| +/-  | 1      | 1/2+1/3+1/2             | 1.333 |
| +/-  | 2      | 1/2+1/2+1/3+1/3+1/2+1/2 | 2.667 |
| +/-  | 3      | 1/2+1/3+1/3+1/2         | 1.667 |
| +/-  | 4      | 1/3                     | 0.333 |

Adopted from Petru Eles, Zebo Peng

Force if op10 is placed in cycle 4:

$$\text{Force}(4) = DG_{\text{mult}}(4) - \text{avg. } DG = 1.0 - 1.167 = \boxed{-0.167}$$

**Interpretation:**

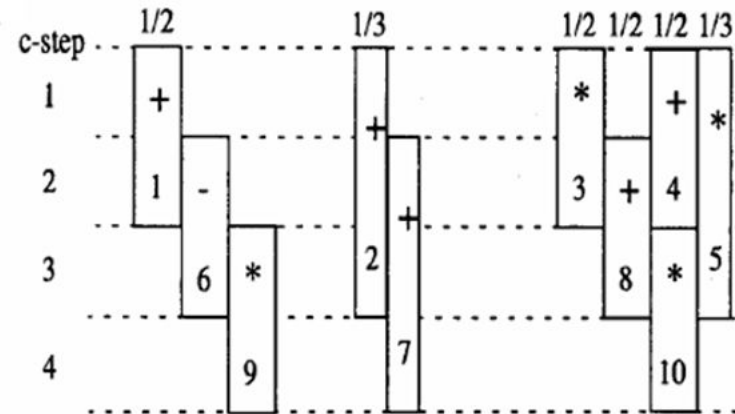
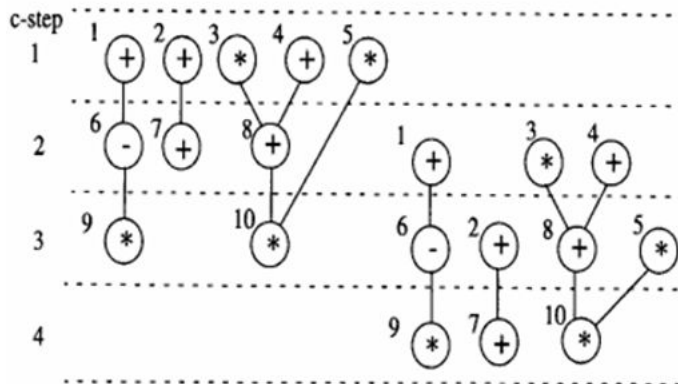
Cycle 4 is **less loaded**

Placing op10 here **improves balance**

# Example: Time-Constrained Scheduling

## Step-05: Operation Schedule.

| C-Step | Operations |   |   |   |   |   |   |   |   |    |
|--------|------------|---|---|---|---|---|---|---|---|----|
|        | 1          | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 1      |            |   |   |   |   |   |   |   |   |    |
| 2      |            |   |   |   |   |   |   |   |   |    |
| 3      |            |   |   |   |   |   |   |   |   |    |
| 4      |            |   |   |   |   |   |   |   |   | ok |



Force if op10 is placed in cycle 4:

$$\text{Force}(4) = DG_{\text{mult}}(4) - \text{avg. } DG = 1.0 - 1.167 = \boxed{-0.167}$$

**Interpretation:**

Cycle 4 is **less loaded**

Placing op10 here **improves balance**

# Time-Constrained Scheduling

- 1) **Start:** all operations are unscheduled.
- 2) **Perform ASAP and ALAP scheduling and compute distribution graphs** for all resources.
- 3) **Calculate all forces** for all unscheduled operations including indirect forces.
- 4) **The pair (operation, control step) with the smallest force** (may be negative!) is selected for scheduling.
- 5) **Update distribution graphs.**
- 6) **Repeat from 3** until all operations are scheduled.

# Exercise-01

- ▶ Solve the differential equation

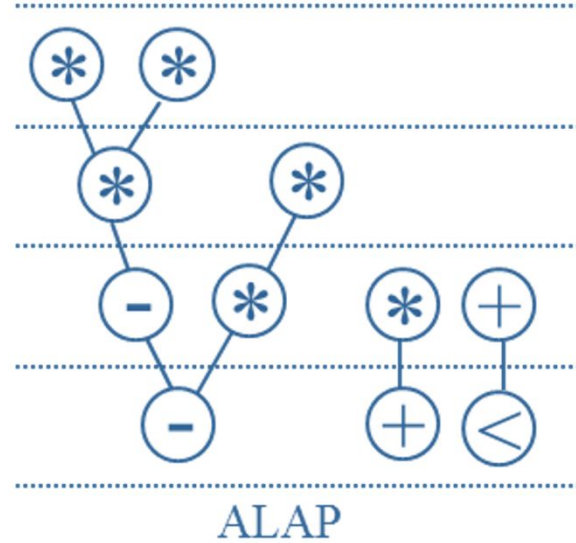
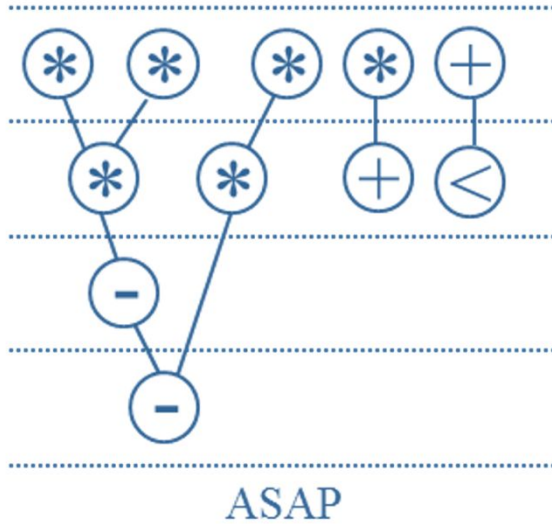
$$\frac{d^2 y}{dx^2} + 3z \frac{dy}{dx} + 3y = 0$$

- ▶ This can be calculated using this iterative algorithm

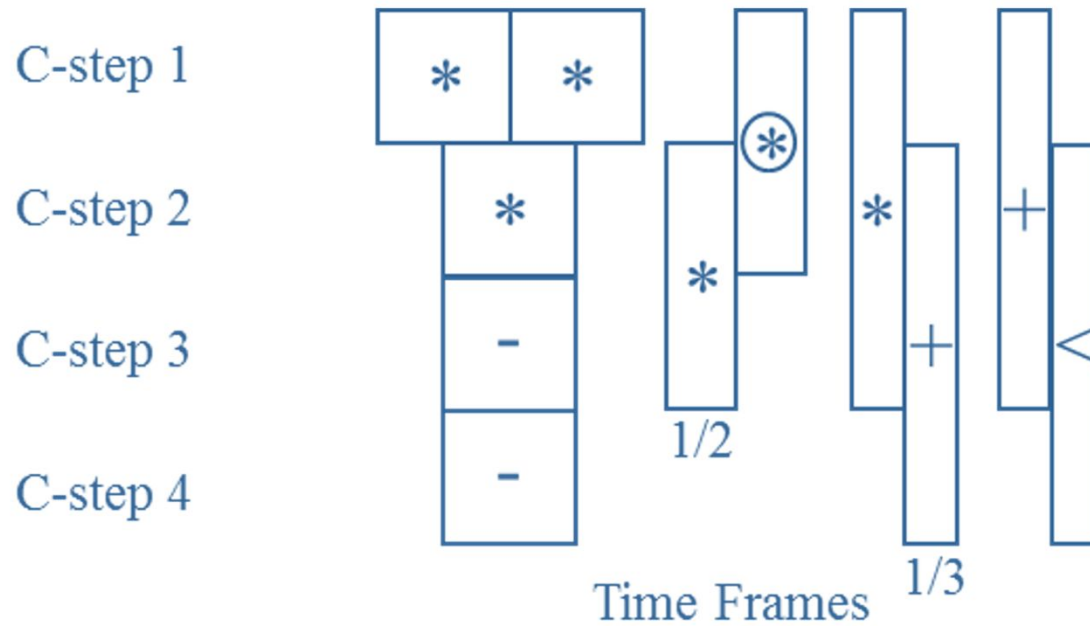
```
while( $z < a$ ) repeat  
   $z1 := z + dz$ ;  
   $ul := u - (3 \cdot z \cdot u \cdot dz) - (3 \cdot y \cdot dz)$ ;  
   $yl := y + (u \cdot dz)$ ;  
   $z := z1$ ;  
   $u := ul$ ;  
   $y := yl$ ;
```

Solve yourself!

# Solution: FDS(1)



## Solution: FDS(2)



# Exercise-02

- ▶ Calculate the DG
- ▶ Design a separate DG table computed for each different resource type.
- ▶ Compute a schedule table by calculating the minimum force of each resource type.

- ▶  $A = A2 * A3$
- ▶  $B = A4 - A5$
- ▶  $O = A + B + A1$
- ▶  $G = A6 * A7$
- ▶  $P = (A10 + G) * A11$
- ▶  $Q = (A8 + A9) * A12$
- ▶ **4-CYCLE POSSIBLE.**

Solve yourself!

# References

1. <http://staff.ustc.edu.cn/~songch/download/An.Introduction.To.HLS.pdf>
2. [http://media.ee.ntu.edu.tw/crash\\_course/2023/2023\\_HLS\\_Introduction\\_and\\_Lab.pdf](http://media.ee.ntu.edu.tw/crash_course/2023/2023_HLS_Introduction_and_Lab.pdf)
3. [https://wiki.dcae.pub.ro/index.php/Lab\\_1\\_-\\_Intro\\_to\\_High\\_Level\\_Synthesis\\_Simulation\\_Co-simulation\\_RTL\\_generation](https://wiki.dcae.pub.ro/index.php/Lab_1_-_Intro_to_High_Level_Synthesis_Simulation_Co-simulation_RTL_generation)
4. [https://users.ece.utexas.edu/~gerstl/ee382v\\_f14/soc/vivado\\_hls/VivadoHLS\\_Overview.pdf](https://users.ece.utexas.edu/~gerstl/ee382v_f14/soc/vivado_hls/VivadoHLS_Overview.pdf)